

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №2
Дисциплина: «Оценка сложности алгоритмов»

Работу выполнил: _____ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Яхонтов А.А.

Краснодар
2026

Тема. Рекурсия.

Цель. Написать рекурсивную процедуру подсчёта вершин в бинарном упорядоченном дереве. Найти функции сложности алгоритма (рекурсивную и не рекурсивную). Вычислить экспериментальную сложность алгоритма. Найти сложность алгоритма с косвенной рекурсией.

Реализуем бинарное упорядоченное дерево:

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def insert_node(root, value):
    if root is None:
        return TreeNode(value)

    if value < root.value:
        root.left = insert_node(root.left, value)
    else:
        root.right = insert_node(root.right, value)
    return root

def generate_tree(values):
    if not values:
        return None

    root = TreeNode(values[0])
    for val in values[1:]:
        insert_node(root, val)
    return root
```

Теперь реализуем метод, рекурсивно подсчитывающий количество вершин в дереве:

```
def recursion_count(node):
    if node is None:
        return 0
    return 1 + recursion_count(node.left) + recursion_count(node.right)
```

Временная сложность:

$T(n) = 1 + T(\frac{n}{2}) + T(\frac{n}{2}) = O(n)$ — функция сложности линейная

Ёмкостная сложность:

$S(n) = n = O(n)$ — функция сложности линейная

Реализуем метод, итеративно подсчитывающий количество вершин в дереве:

```
def iteration_count(root):
    if root is None:
        return 0

    count = 0
    stack = [root]

    while stack:
        node = stack.pop()
        count += 1
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)

    return count
```

Временная сложность:

$T(n) = 4n + C = O(n)$ — функция сложности линейная

Ёмкостная сложность:

$S(n) = n = O(n)$ — функция сложности линейная

Затем реализуем алгоритм, подсчитывающий количество вершин в дереве с помощью косвенной рекурсии:

```
def count_vertex_helper1(node):
    if node is None:
        return 0
    return count_vertex_helper2(node.right) + count_vertex_helper2(node.left) + 1

def count_vertex_helper2(node):
    if node is None:
        return 0
    return 1 + count_vertex_helper1(node.left) + count_vertex_helper1(node.right)

def indirect_count(root):
    return count_vertex_helper1(root)
```

Временная сложность:

$$T1(n) = T2\left(\frac{n}{2}\right) + T2\left(\frac{n}{2}\right) + 1$$

$$T2(n) = 1 + T1\left(\frac{n}{2}\right) + T1\left(\frac{n}{2}\right)$$

$$T(n) = T1(n) = 1 + T\left(\frac{n}{4}\right) + T\left(\frac{n}{4}\right) + 1 + T\left(\frac{n}{4}\right) + T\left(\frac{n}{4}\right) + 1 = O(n) \text{ — функция}$$

сложности линейная

Ёмкостная сложность:

$S(n) = n = O(n)$ — функция сложности линейная

Экспериментальная сложность алгоритмов:

